

A camera, a Linux box, and one event into your CMS.

The two items from our call on 18 September: what the hardware has to be, and how the segment reaches your player.

● Minimum HW requirements

● Send the triggering event for CMS

SECTION 1 • MINIMUM HW REQUIREMENTS

One Linux box, one camera.

Commodity parts, both sourceable in China.

MEDIA PLAYER / EDGE NODE	
Raspberry Pi 5, 4 GB	
OS	Raspberry Pi OS 64-bit, Debian based
CPU	4 cores, Cortex-A76 class, 2.4 GHz
RAM	4 GB
Storage	64 GB microSD
Power and cooling	27 W USB-C PD. Active cooling required
Ports	1 free USB 2.0 for the camera
Network	Ethernet or Wi-Fi. Optional
Accelerator	None. No NPU or GPU needed

CAMERA	
RedEagle 2K USB webcam	
Interface	USB, UVC class, no vendor driver
Resolution	1080p minimum, 2K in our units
Lens	2.8 mm, fixed focus
Field of view	120 degrees
Placement	Outside the screen, on the shelf, facing the approach zone
Unit cost	Around EUR 20

SCOPE OF THESE FIGURES

Our application alone on the box. If a media player runs on the same hardware, its CPU, memory and thermal budget come on top of this.

SOURCING

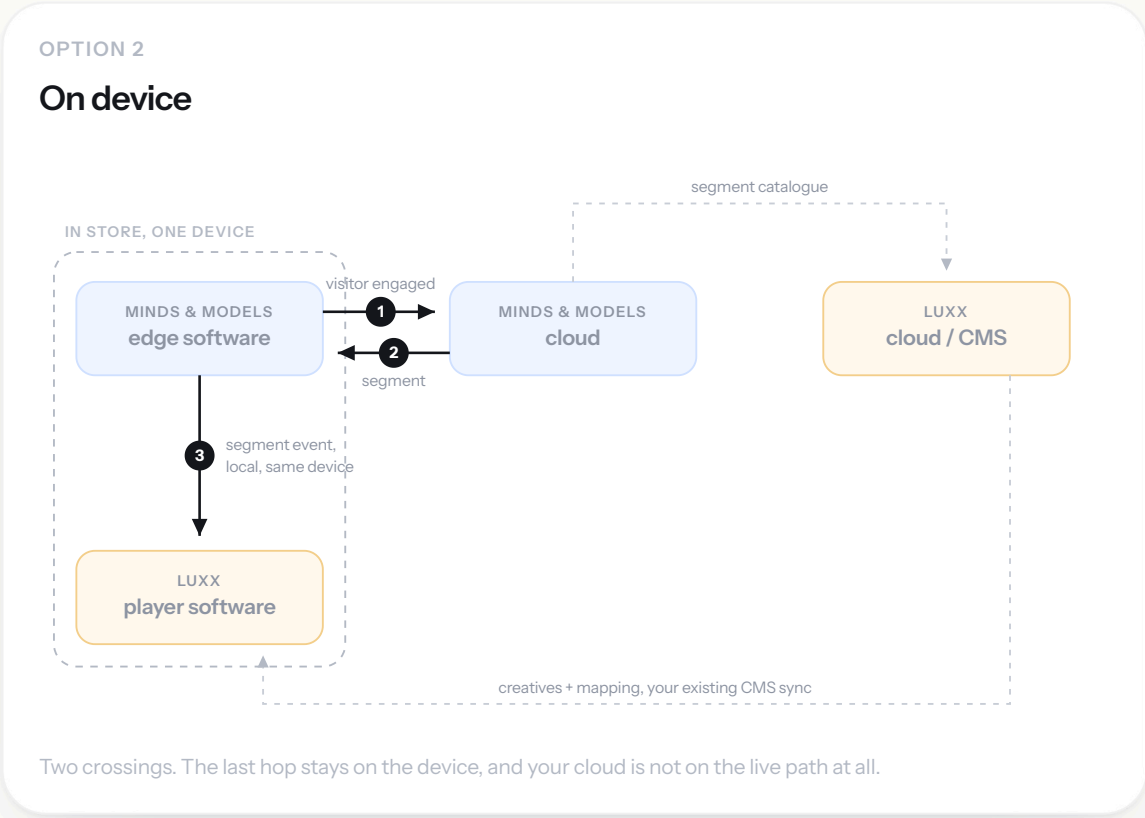
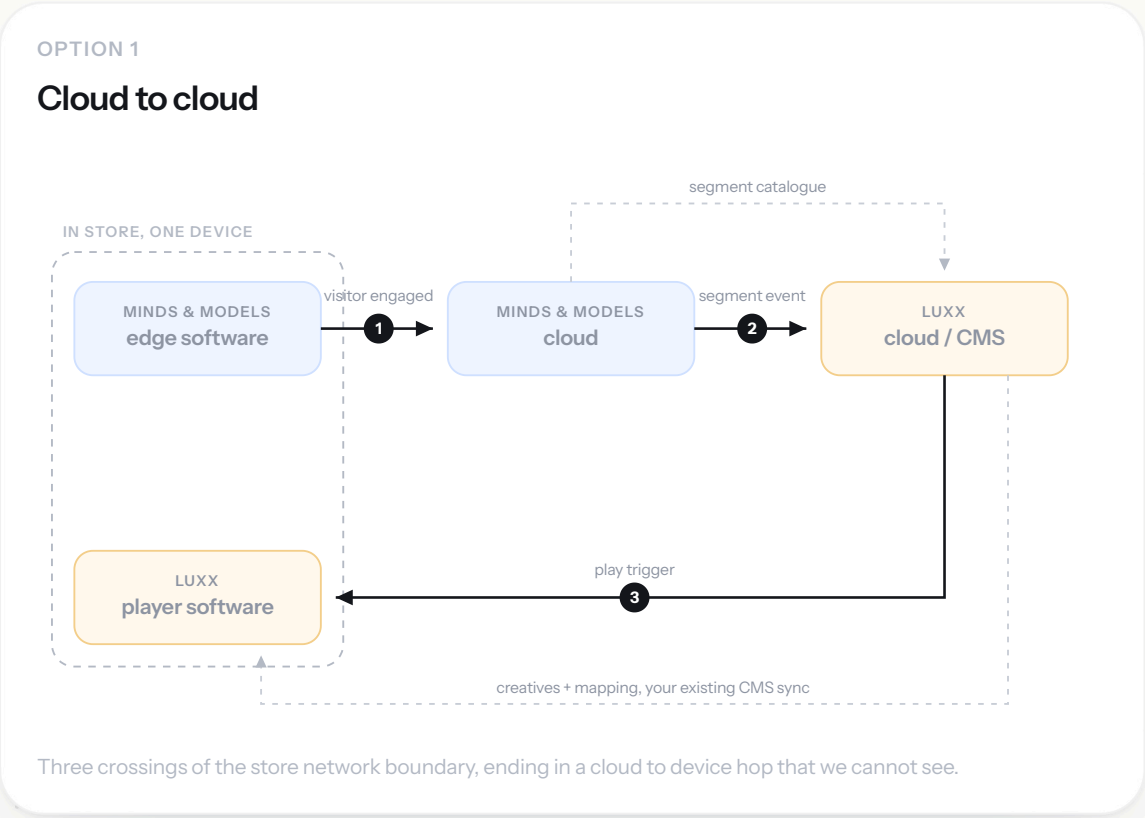
Any comparable UVC camera works once we have tested it. Accuracy is decided by mounting geometry, not by sensor price. LUXX can source both parts in China.

SECTION 2 · SEND THE TRIGGERING EVENT FOR CMS

Two ways to hand you the segment.

Same event, same fields. Only the delivery differs.

■ Minds & Models component ■ LUXX component □ One physical device in store, shared hardware → Solid arrow: the live path, while the shopper is still there ··· Dotted: configuration, identical in both



WHAT EACH OPTION ASKS OF YOU

Option 1 changes your backend.

Option 2 changes every installed screen.

OPTION 1 · CLOUD TO CLOUD

What changes

- ✓ An HTTPS endpoint that receives the event.
- ✓ **A live push from your cloud to one specific player**, while the shopper is still there.
- ✓ Resolving `edge_node_id` to a player, and a segment to creative mapping in your CMS.

What it costs

- Two networks in the live path. A slow or unreachable push degrades quietly to the fallback.
- No shared latency budget. Neither side can measure the whole chain alone.

ANSWER THIS FIRST

How fast can your cloud reach one specific player? If you already push live, this is mostly plumbing. If it is a poll or a CMS sync, that is where the effort sits.

OPTION 2 · ON DEVICE

What changes

- ✓ Your player accepts a local event and resolves it to a creative.
- ✓ Your backend gets the segment to creative mapping down to devices, alongside the media.
- ✓ **Both halves handle a segment they have no creative for.**

What it costs

- Your player has to be extensible. A closed appliance will not do.
- Every installed screen needs a player update, and again on every protocol change. Our release trains stay coupled.

IN EXCHANGE

Latency stops being your problem. The trigger never leaves the device, so time to display is ours to control, measurable, and something we can commit to.

SIDE BY SIDE

The same nine questions,
answered two different ways.

	OPTION 1 · CLOUD TO CLOUD	OPTION 2 · ON DEVICE
Where the trigger is delivered	Your backend	Your player, on the same device
Changes to your player software	None	Required
Changes to your backend	Receive our event, push it to the right player, live	Get the segment to creative mapping onto devices
What you deploy to installed screens	Nothing	A player update, and again on every protocol change
New real-time cloud to device channel	Required unless you already have one	Not required
Who controls end-to-end latency	Shared. Your cloud to player hop sets it	Mostly us. The final hop is local
Release coordination between us	None. Each side ships on its own schedule	Protocol changes ship on both sides together
A mixed player fleet	Tolerated. Only the backend changes	Depends on each player being extensible
How it fails quietly	A slow or unreachable push path	A device whose configuration sync has lagged

THE EVENT

One session, two events.

Two fields carry the decision, the rest is context.

A **session** is one shopper's engagement with a screen. It begins when someone gives the screen sustained attention and ends when they leave. It produces exactly two events: a start that carries the segment, and an end.

FIELD	TYPE	WHY IT IS THERE
segment_id REQ	uuid	The decision. The only field you need to pick a creative. Absent on session_end.
edge_node_id REQ	uuid	Which of our devices it came from. Your handle for pairing to a player.
event REQ	enum	session_start or session_end.
event_id REQ	string	Idempotency, and correlating your logs with ours.
occurred_at REQ	timestamp	Lets you drop an event that arrived too late to be worth showing.
segment_name	string	Human readable. What you do with it is up to you.
age_band	string	A coarse band, for example 30s. Never an exact age.
gender	string	A coarse estimate, useful as a secondary key if you want one.

ILLUSTRATIVE. EXACT SHAPE TO BE AGREED

```
{
  "event":      "session_start",
  "event_id":   "9f1c4a2e-7b30-4d51-9a6e-0c2b",
  "occurred_at": "2026-09-18T10:41:22.418Z",
  "edge_node_id": "3b7a1f60-4c22-4f0e-9d18-6a5c",
  "segment_id":  "a41d90c7-2e58-4b16-8f03-51d7",
  "segment_name": "Weekend family shopper",
  "age_band":    "30s",
  "gender":      "F"
}
```

// the end of the same session

```
{ "event": "session_end", "event_id": "c0d5",
  "occurred_at": "...10:41:29.902Z",
  "edge_node_id": "3b7a1f60-4c22..." }
```

PAIRING OUR DEVICE TO YOUR PLAYER

Every event carries edge_node_id, and our API publishes where each node is installed. Enough for you to pair it with the right player. **In Option 2 the event is already on the right device.**

TRUE IN BOTH OPTIONS

What is the same either way.

Including the one detail that is expensive to find in production.

BOTH OPTIONS NEED

- ✓ **A camera on every participating screen**, facing the shopper. A hardware and mounting change, and an extra step at installation.
- ✓ **Two companies' software on one box**. CPU, memory and thermal budget, the OS baseline, who owns updates, what the watchdog does.
- ✓ **The CMS stays yours**. What plays where and when, how media reaches the device, how it is cached.
- ✓ **A segment catalogue on our API**. We publish each retail customer's segments, you read them and keep your mapping in step.
- ✓ **A stable ID and a location for every edge node**, so you can pair it with the right player.

🛡️ **Frames are processed on the device and not retained.** What leaves is anonymized: no identifiable imagery, no biometric template, no re-identification. **The system classifies, it does not recognize.**

THE DETAIL WORTH STATING PLAINLY

The segment catalogue changes over time

- + Segments are derived **per retail customer**, from that customer's own visitor data. Not a fixed global taxonomy.
- + When a new segmentation is published, the catalogue is re-issued and **segment IDs are not guaranteed to survive it**.
- + We expose a catalogue version and a published-at timestamp, so your CMS can flag segments with no creative behind them.

An unknown or unmapped segment ID means "stay on the fallback", never an error. Worth agreeing that explicitly.

WHERE THIS GOES NEXT

Six things to agree, and one place we would start.

OPEN POINTS

- The exact payload, and which optional fields you actually want.
- Transport and authentication for the handoff.
- Session-end semantics, and the fallback behavior on your side.
- Catalogue versioning, and what your CMS does with an unmapped segment.
- What each side measures for latency, and what each of us will commit to.
- Pilot scope: how many stores, for how long, and what we judge it by.

OUR SUGGESTION

Start with Option 1

It is the smaller change for both of us. **Your player software, your release process and your installed screens stay as they are**, and the whole integration lives in one backend on each side.

The one thing to establish first is how quickly your cloud can reach a specific player. If that path already exists, this is largely plumbing.

AND IT IS NOT A ONE-WAY DOOR

Option 2 is the answer if that check goes the other way. **The event is identical in both: same fields, same meaning, only delivered somewhere else.** Moving later is a change of delivery, not a redesign.